

Lab 3 – Attacchi

Sicurezza dei Sistemi e delle reti

Modulo 3 - Autenticazione e controllo degli accessi

Unità didattica 3 – Controllo degli accessi nei sistemi operativi

Stelvio Cimato

Università degli Studi di Milano - SSRI - CdL Online

Attacchi tramite input dell'utente

- **Input dell'utente : input espliciti**
 - Buffer Overflow
 - Overflow di un buffer per eseguire codice dannoso
 - Vulnerabilità delle stringhe di formato:
 - Modifica del comportamento del programma utilizzando gli input dell'utente come stringhe di formato

Attacchi tramite input di sistema

- **Ingressi di sistema**

- Race Condition
- Collegamento simbolico al file privilegiato da un file non privilegiato
 - Influenza i Programmi
 - Scrittura all'interno della cartella scrivibile da tutti

Attacchi tramite variabili d'ambiente

- **Il comportamento può essere influenzato da input che non sono visibili all'interno di un programma.**
- **Variabili d'ambiente : possono essere impostate da un utente prima di eseguire un programma.**

Attacchi tramite variabili d'ambiente

- **PATH Variable**

- Utilizzato dai programmi shell per individuare un comando se l'utente non fornisce il percorso completo per il comando
- `system()`: chiama prima `/bin/`
- sistema ("`ls`")
 - `/bin/sh` utilizza la variabile d'ambiente `PATH` per individuare "`ls`"
 - L'attaccante può manipolare la variabile `PATH` e controllare come viene trovato il comando "`ls`".

Perdita di capacità

- **In alcuni casi, i programmi privilegiati si autodeclassano durante l'esecuzione**
- **Esempio: il su programma**
 - Questo è un programma Set-UID privilegiato
 - Consente a un utente di passare a un altro utente (ad esempio da utente1 a utente2)
 - Il programma inizia con EUID come root e RUID come utente1
 - Dopo la verifica della password, sia EUID che RUID diventano dell'utente2 (tramite downgrade dei privilegi)
- **Tali programmi potrebbero portare a perdite di capacità**
 - I programmi potrebbero non eliminare le funzionalità privilegiate prima del downgrade

Attacchi tramite perdita di capacità: un esempio

Il file /etc/ zzz è scrivibile solo da root

Viene creato il descrittore di file
(il programma è un programma Set-UID di proprietà di root)

Il privilegio viene declassato

Invoca un programma di shell, quindi la restrizione di comportamento sul programma viene eliminata

```
fd = open("/etc/zzz", O_RDWR | O_APPEND);
if (fd == -1) {
    printf("Cannot open /etc/zzz\n");
    exit(0);
}

// Print out the file descriptor value
printf("fd is %d\n", fd);

// Permanently disable the privilege by making the
// effective uid the same as the real uid
setuid(getuid());

// Execute /bin/sh
v[0] = "/bin/sh"; v[1] = 0;
execve(v[0], v, 0);
```

Attacchi tramite perdita di capacità (continua)

Il programma dimentica di chiudere il file, quindi il descrittore di file è ancora valido.



Perdita di capacità

```
$ gcc -o cap_leak cap_leak.c
$ sudo chown root cap_leak
[sudo] password for seed:
$ sudo chmod 4755 cap_leak
$ ls -l cap_leak
-rwsr-xr-x 1 root seed 7386 Feb 23 09:24 cap_leak
$ cat /etc/zxx
bbbbbbbbbbbbbbbbbb
$ echo aaaaaaaaaa > /etc/zxx
bash: /etc/zxx: Permission denied ← Cannot write to the file
$ cap_leak
fd is 3
$ echo cccccccccccc >& 3 ← Using the leaked capability
$ cat /etc/zxx
bbbbbbbbbbbbbbbbbb
cccccccccccccc ← File modified
```

Come sistemare il programma?

Distruggi il descrittore di file prima di declassare il privilegio (chiudi il file)

Perdita di capacità in OS X: caso di studio

- **OS X Yosemite è stato ritenuto vulnerabile all'attacco di escalation dei privilegi correlato alla perdita di capacità nel luglio 2015 (OS X 10.10)**
- **Aggiunte funzionalità al linker dinamico dyld**
 - Variabile di ambiente DYLD_PRINT_TO_FILE
- **Il linker dinamico può aprire qualsiasi file, quindi per i programmi Set-UID di proprietà di root viene eseguito con privilegi di root. Il linker dinamico dyld non chiude il file. C'è una perdita di capacità .**
- **Scenario 1 (sicuro): Set-UID ha terminato il suo lavoro e il processo muore. Tutto è ripulito ed è sicuro.**
- **Scenario 2 (non sicuro): simile al programma " su ", il programma privilegiato riduce i suoi privilegi e revoca la restrizione.**

Invocare dei programmi

- **Invocazione di comandi esterni dall'interno di un programma**
- **Il comando esterno viene scelto dal programma Set-UID**
 - Gli utenti non dovrebbero fornire il comando (o non è sicuro)
- **Attacco:**
 - Agli utenti viene spesso chiesto di fornire dati di input al comando.
 - Se il comando non viene richiamato correttamente, i dati di input dell'utente potrebbero essere trasformati nel nome del comando. Questo è pericoloso.

Richiamo di programmi: approccio non sicuro

```
int main(int argc, char *argv[])
{
    char *cat="/bin/cat";

    if(argc < 2) {
        printf("Please type a file name.\n");
        return 1;
    }

    char *command = malloc(strlen(cat) + strlen(argv[1]) + 2);
    sprintf(command, "%s %s", cat, argv[1]);
    system(command);
    return 0 ;
}
```

- Il modo più semplice per invocare un comando esterno è la funzione **system()**.
- Questo programma dovrebbe eseguire il programma **/bin/cat** .
- È un programma Set-UID di proprietà di root, quindi il programma può visualizzare tutti i file, ma non può scrivere su nessun file.

Domanda: puoi usare questo programma per eseguire altri comandi, con i privilegi di root?

Richiamo di programmi: approccio non sicuro (continua)

- Possiamo ottenere una shell di root con questo input

```
$ gcc -o catall catall.c
$ sudo chown root catall
$ sudo chmod 4755 catall
$ ls -l catall
-rwsr-xr-x 1 root seed 7275 Feb 23 09:41 catall
$ catall /etc/shadow
root:$6$012BPz.K$fbPkT6H6Db4/B8cLWb....
daemon*:15749:0:99999:7:::
bin*:15749:0:99999:7:::
sys*:15749:0:99999:7:::
sync*:15749:0:99999:7:::
games*:15749:0:99999:7:::

$ catall "aa;/bin/sh"
/bin/cat: aa: No such file or directory
#      ← Got the root shell!
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=0(root), ...
```



Problema :
alcune parti dei
dati diventano
codice (nome
del comando)

Una nota

- **In Ubuntu 16.04, /bin/ sh punta a /bin/dash, che ha una contromisura**
 - Perde i privilegi quando viene eseguito all'interno di un processo set-uid
- **Pertanto, nell'attacco della diapositiva precedente otterremo solo una shell normale**
- **Effettuare le seguenti operazioni per rimuovere la contromisura**

```
Before experiment: link /bin/sh to /bin/zsh  
$ sudo ln -sf /bin/zsh /bin/sh
```

```
After experiment: remember to change it back  
$ sudo ln -sf /bin/dash /bin/sh
```

Richiamo sicuro dei programmi: utilizzo di `execve()`

```
int main(int argc, char *argv[])
{
    char *v[3];

    if(argc < 2) {
        printf("Please type a file name.\n");
        return 1;
    }

    v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = 0;
    execve(v[0], v, 0);

    return 0 ;
}
```

`execve(v[0], v, 0)`

Il nome del comando viene fornito qui (dal programma)

I dati di input sono forniti qui (possono essere forniti dall'utente)

Perché è sicuro?

Il codice (nome del comando) e i dati sono chiaramente separati; non è possibile che i dati dell'utente diventino codice

Richiamo sicuro dei programmi (continua)

```
$ gcc -o safecatall safecatall.c
$ sudo chown root safecatall
$ sudo chmod 4755 safecatall
$ safecatall /etc/shadow
root:$6$012BPz.K$fbPkT6H6Db4/B8cLWb....
daemon*:15749:0:99999:7:::
bin*:15749:0:99999:7:::
sys*:15749:0:99999:7:::
sync*:15749:0:99999:7:::
games*:15749:0:99999:7:::

$ safecatall "aa;/bin/sh"
/bin/cat: aa;/bin/sh: No such file or directory ← Attack failed!
```



I dati vengono ancora trattati come dati, non come codice

Considerazione aggiuntiva

- **Alcune funzioni nella famiglia `exec()` si comportano in modo simile a `execve()`, ma potrebbero non essere sicure**
 - `execlp()`, `execvp()` e `execvpe()` duplicano le azioni della shell. Queste funzioni possono essere attaccate utilizzando la variabile d'ambiente `PATH`

Richiamo di comandi esterni in altri linguaggi

- **Il rischio di invocare comandi esterni non è limitato ai programmi C**
- **Dovremmo evitare problemi simili a quelli causati dalle funzioni `system()`**
- **Esempi:**
 - Perl: `open ()` può eseguire comandi, ma lo fa attraverso una shell
 - PHP: funzione di sistema `()`.

```
<?php
print("Please specify the path of the directory");
print("<p>");
$dir=$_GET['dir'];
print("Directory path: " . $dir . "<p>");
system("/bin/ls $dir");
?>
```

- Comando eseguito sul server: `/bin/ls ./date`

Principio di isolamento

- **Principio: non mescolare codice e dati.**
- **Attacchi dovuti alla violazione di questo principio:**
 - `system ()` .
 - Cross Site Scripting – Ulteriori informazioni nel capitolo 10
 - SQL injection - Ulteriori informazioni nel capitolo 11
 - Attacchi Buffer Overflow: ulteriori informazioni nel capitolo 4

Principio del privilegio minimo

- **A un programma privilegiato dovrebbe essere dato il potere necessario per svolgere i suoi compiti.**
- **Disabilitare i privilegi (temporaneamente o permanentemente) quando un programma privilegiato non ne ha bisogno .**
- **In Linux, seteuid () e setuid () possono essere utilizzati per disabilitare/annullare i privilegi.**
- **Sistemi operativi diversi hanno modi diversi per farlo.**